

Assignment 15.3 Ai Assisted Coding

Htno:2303a51305

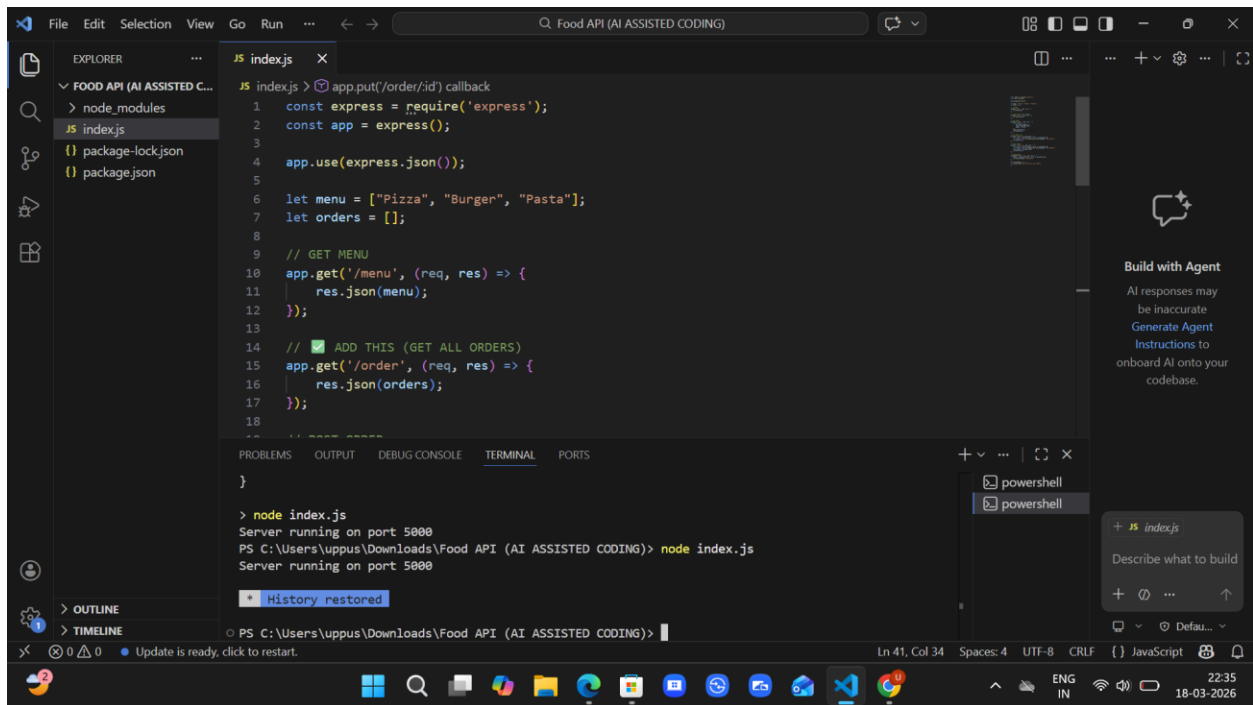
Btno:05

Task 1: Online Food Ordering API

Prompt:

This task involves developing a RESTful API for an online food ordering system. The API allows users to view available menu items, place orders, track order status, update orders, and cancel orders. It uses different HTTP methods like GET, POST, PUT, and DELETE to perform these operations. AI tools are used to assist in generating and improving the backend code.

Code:

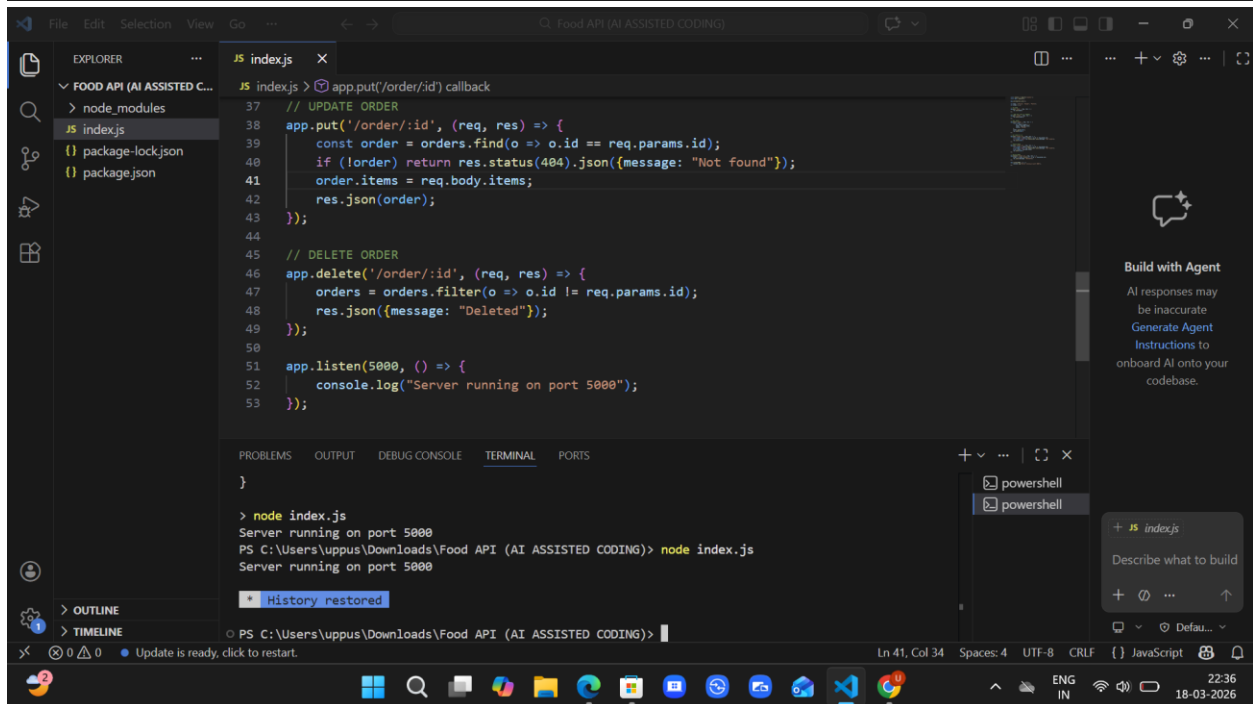


VS Code editor showing the initial setup of a Node.js application. The Explorer sidebar shows the project structure with 'index.js' selected. The main editor displays the code for 'index.js' with comments and a terminal window at the bottom showing the command 'node index.js' and the server running on port 5000.

```
1 const express = require('express');
2 const app = express();
3
4 app.use(express.json());
5
6 let menu = ["Pizza", "Burger", "Pasta"];
7 let orders = [];
8
9 // GET MENU
10 app.get('/menu', (req, res) => {
11   res.json(menu);
12 });
13
14 // ADD THIS (GET ALL ORDERS)
15 app.get('/order', (req, res) => {
16   res.json(orders);
17 });
18
19 // ...
20 }
```

Terminal output:

```
> node index.js
Server running on port 5000
PS C:\Users\uppus\Downloads\Food API (AI ASSISTED CODING)> node index.js
Server running on port 5000
```



VS Code editor showing the updated code for 'index.js'. The code now includes 'UPDATE ORDER' and 'DELETE ORDER' endpoints. The terminal window shows the command 'node index.js' and the server running on port 5000.

```
37 // UPDATE ORDER
38 app.put('/order/:id', (req, res) => {
39   const order = orders.find(o => o.id == req.params.id);
40   if (!order) return res.status(404).json({message: "Not found"});
41   order.items = req.body.items;
42   res.json(order);
43 });
44
45 // DELETE ORDER
46 app.delete('/order/:id', (req, res) => {
47   orders = orders.filter(o => o.id != req.params.id);
48   res.json({message: "Deleted"});
49 });
50
51 app.listen(5000, () => {
52   console.log("Server running on port 5000");
53 });
```

Terminal output:

```
> node index.js
Server running on port 5000
PS C:\Users\uppus\Downloads\Food API (AI ASSISTED CODING)> node index.js
Server running on port 5000
```

Output:

uppu snehanjali's Workspace

Search

GET Get data GET Untitled Request POST http://localhost:5000/

http://localhost:5000/order

POST http://localhost:5000/order

Docs Params Authorization Headers 8 Body Scripts Tests Settings

none form-data x-www-form-urlencoded raw binary GraphQL JSON

```
1 {
2   "items": ["Pizza"]
3 }
```

Body Cookies Headers 7 Test Results

200 OK · 39 ms · 279 B

{ } JSON

```
1 {
2   "id": 1,
3   "items": [
4     "Pizza"
5   ],
6   "status": "Placed"
7 }
```

ENVIRONMENTS SPECS FLOWS

Connect Git Console Terminal

19:18 18-03-2026

uppu snehanjali's Workspace

Search

PUT http: DEL http: GET http: GET http: POST Unt POST htt PUT http: DEL http: POST htt POST htt

http://localhost:5000/order

POST http://localhost:5000/order

Docs Params Authorization Headers 8 Body Scripts Tests Settings

none form-data x-www-form-urlencoded raw binary GraphQL JSON

```
1 {
2   "item": "Pizza",
3   "quantity": 2
4 }
```

Body Cookies Headers 7 Test Results

200 OK · 31 ms · 261 B

{ } JSON

```
1 {
2   "id": 1,
3   "status": "Placed"
4 }
```

ENVIRONMENTS SPECS FLOWS

Connect Git Console Terminal

22:55 18-03-2026

uppu snehanjali's Workspace

Search

DEL http: GET http: GET http: POST Unt POST htt PUT http: DEL http: POST htt POST htt GET http: + No enviro...

http://localhost:5000/order/1

GET http://localhost:5000/order/1

Send

Docs Params Authorization Headers 6 Body Scripts Tests Settings Cookies

Query Params

Key	Value	Description	Bulk Edit
Key	Value	Description	

Body Cookies Headers 7 Test Results

200 OK · 4 ms · 261 B

JSON Preview Visualize

```
1 {
2   "id": 1,
3   "status": "Placed"
4 }
```

ENVIRONMENTS

SPECS

FLOWS

Connect Git Console Terminal

Globals Vault Tools

22:56 18-03-2026

uppu snehanjali's Workspace

Search

GET http: GET http: POST Unt POST htt PUT http: DEL http: POST htt POST htt GET http: PUT http: + No enviro...

http://localhost:5000/order/1

PUT http://localhost:5000/order/1

Send

Docs Params Authorization Headers 8 Body Scripts Tests Settings Cookies

none form-data x-www-form-urlencoded raw binary GraphQL JSON Schema Beautify

```
1 {
2   "quantity": 3
3 }
```

Body Cookies Headers 7 Test Results

200 OK · 5 ms · 261 B

JSON Preview Visualize

```
1 {
2   "id": 1,
3   "status": "Placed"
4 }
```

ENVIRONMENTS

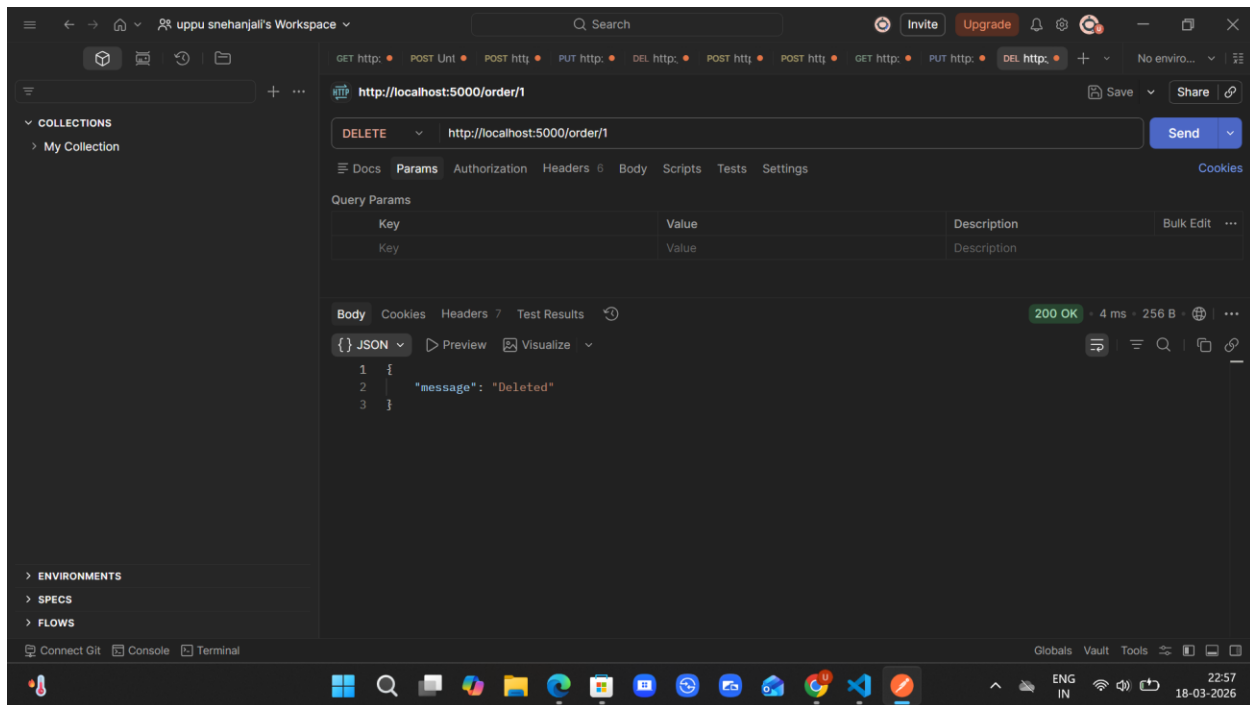
SPECS

FLOWS

Connect Git Console Terminal

Globals Vault Tools

22:56 18-03-2026



Expalantion:

-->This task is about creating a food ordering system API where users can interact with menu and orders.

-->Users can view available food items using the GET method.

-->They can place a new order using the POST method by sending item and quantity details.

-->The system allows updating order details using PUT and cancelling orders using DELETE.

-->Overall, this task helps in understanding how real-world applications use APIs to manage data.

Task 2: Employee Payroll API

Prompt:

This task involves building a RESTful API to manage employee details and their salaries. The API allows adding new employees, viewing employee data, updating salary information, and deleting employees. It uses HTTP methods like GET, POST, PUT, and DELETE. The goal is to create a structured and secure payroll management system.

Code:

uppu snehanjali's Workspace

Search

POST http://localhost:3000/employees

GET http://localhost:3000/employees

Send

Docs Params Authorization Headers 6 Body Scripts Tests Settings

Query Params

Key	Value	Description	Bulk Edit
Key	Value	Description	

Body Cookies Headers 7 Test Results

200 OK · 20 ms · 235 B

JSON Preview Visualize

```
1 {}
```

ENVIRONMENTS SPECS FLOWS

Connect Git Console Terminal

Windows Taskbar: 23:02 18-03-2026

uppu snehanjali's Workspace

Search

POST http://localhost:3000/employees

POST

Send

Docs Params Authorization Headers 8 Body Scripts Tests Settings

Body Cookies Headers 7 Test Results

200 OK · 34 ms · 272 B

JSON Preview Visualize

```
1 {
2   "name": "Ravi",
3   "salary": 30000
4 }
5 {
6   "id": 1,
7   "name": "Ravi",
8   "salary": 30000
9 }
```

ENVIRONMENTS SPECS FLOWS

Connect Git Console Terminal

Windows Taskbar: 23:03 18-03-2026

uppu snehanjali's Workspace

Search

PUT http: • DEL http: • POST htt: • POST htt: • GET http: • PUT http: • DEL http: • GET http: • POST htt: • GET http: • + No enviro...

http://localhost:3000/employees

GET http://localhost:3000/employees

Send

Docs Params Authorization Headers 6 Body Scripts Tests Settings Cookies

Query Params

Key	Value	Description	Bulk Edit
Key	Value	Description	

Body Cookies Headers 7 Test Results

200 OK • 4 ms • 274 B

JSON Preview Visualize

```
1 [
2   {
3     "id": 1,
4     "name": "Ravi",
5     "salary": 30000
6   }
7 ]
```

ENVIRONMENTS SPECS FLOWS

Connect Git Console Terminal

Globals Vault Tools

ENG IN 23:04 18-03-2026

uppu snehanjali's Workspace

Search

DEL http: • POST htt: • POST htt: • GET http: • PUT http: • DEL http: • GET http: • POST htt: • GET http: • PUT http: • + No enviro...

http://localhost:3000/employees/1/salary

PUT http://localhost:3000/employees/1/salary

Send

Docs Params Authorization Headers 8 Body Scripts Tests Settings Cookies

none form-data x-www-form-urlencoded raw binary GraphQL JSON Schema Beautify

```
1 {
2   "salary": 50000
3 }
```

Body Cookies Headers 7 Test Results

200 OK • 5 ms • 272 B

JSON Preview Visualize

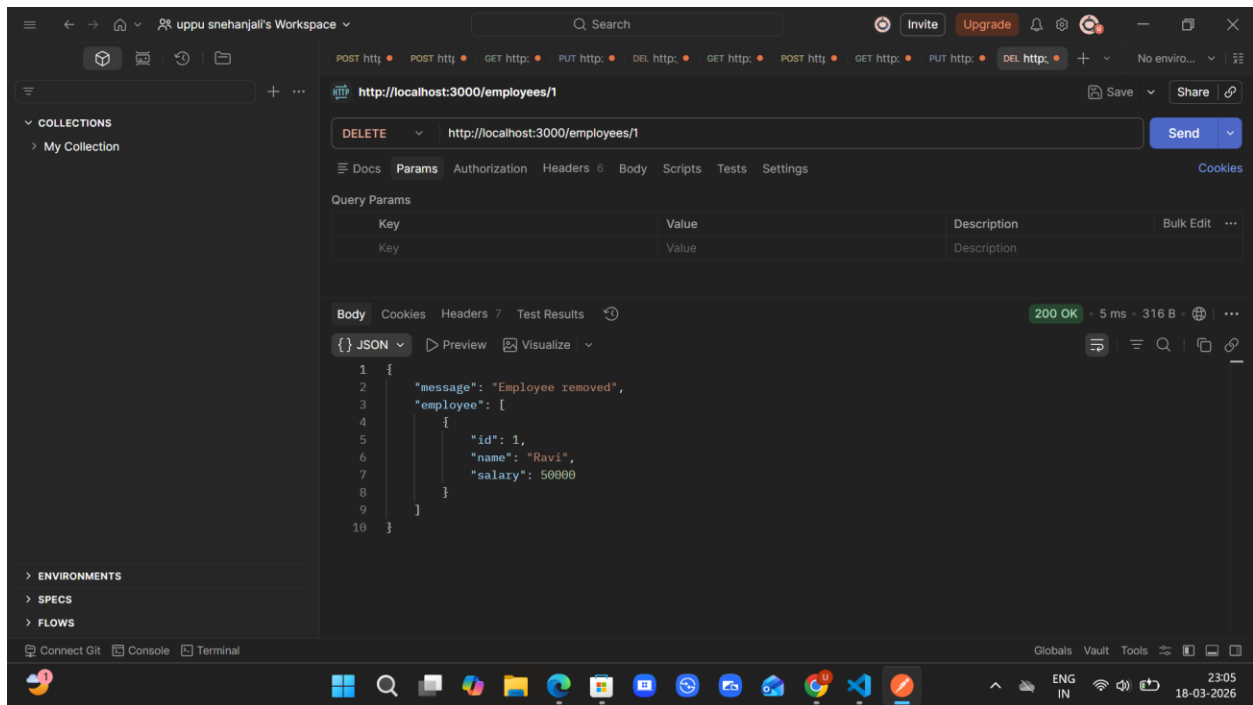
```
1 {
2   "id": 1,
3   "name": "Ravi",
4   "salary": 50000
5 }
```

ENVIRONMENTS SPECS FLOWS

Connect Git Console Terminal

Globals Vault Tools

ENG IN 23:05 18-03-2026



Explanation:

- >This task is about creating an API to manage employee and salary details.
- >Employees can be added using the POST method.
- >All employee data can be viewed using the GET method.
- >Salary can be updated using PUT and employees can be removed using DELETE.
- >This helps in understanding how APIs manage real-world payroll systems.

Task 3: Student Information REST API

Prompt:

This task involves building a RESTful API to manage student records in a university system. The API allows adding new students, viewing student details, updating information, and deleting records. It uses HTTP methods like GET, POST, PUT, and DELETE. The goal is to create a simple and efficient student management system.

Code:

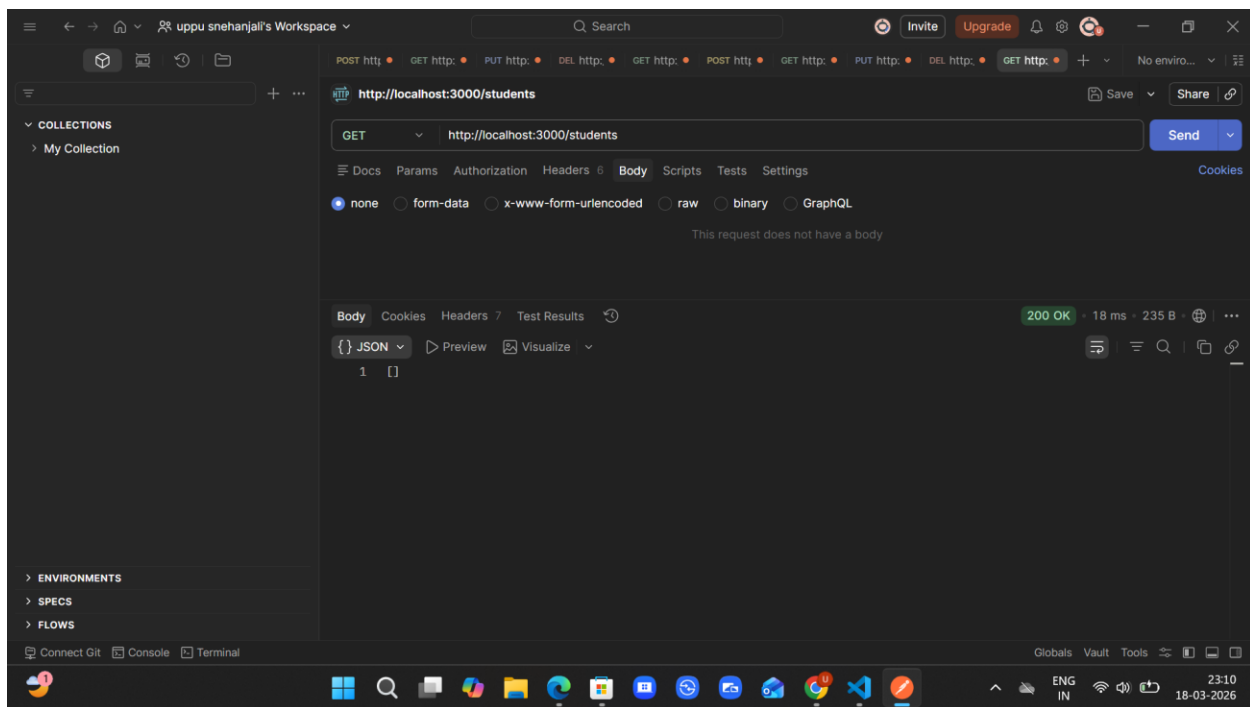
```
1 const express = require('express');
2 const app = express();
3
4 // Middleware to read JSON
5 app.use(express.json());
6
7 // Data storage (array)
8 let students = [];
9
10
11 // ===== GET (View All Students) =====
12 app.get('/students', (req, res) => {
13   res.json(students);
14 });
15
16
17 // ===== POST (Add Student) =====
18 app.post('/students', (req, res) => {
19   // ...
20 });
```

PS C:\Users\uppus\Downloads\student-api-js> node index.js
Server running on port 3000

```
59
60 // ===== DELETE (Remove Student) =====
61 app.delete('/students/:id', (req, res) => {
62   const id = parseInt(req.params.id);
63
64   const index = students.findIndex(s => s.id === id);
65
66   if (index === -1) {
67     return res.status(404).json({ message: "Student not found" });
68   }
69
70   const removed = students.splice(index, 1);
71
72   res.json({
73     message: "Student deleted",
74     student: removed
75   });
76 });
```

PS C:\Users\uppus\Downloads\student-api-js> node index.js
Server running on port 3000

Output:



uppu snehanjali's Workspace

Search

Invites Upgrade

GET http: PUT http: DEL http: GET http: POST http: GET http: PUT http: DEL http: GET http: POST http: + No enviro...

http://localhost:3000/students

POST http://localhost:3000/students

Send

Docs Params Authorization Headers 8 Body Scripts Tests Settings Cookies

none form-data x-www-form-urlencoded raw binary GraphQL JSON Schema Beautify

```
3 {
4   "roll": "23CS001",
5   "course": "CSE",
6   "year": 3
7 }
```

Body Cookies Headers 7 Test Results

200 OK · 65 ms · 299 B

JSON Preview Visualize

```
1 {
2   "id": 1,
3   "name": "Sneha",
4   "roll": "23CS001",
5   "course": "CSE",
6   "year": 3
7 }
```

ENVIRONMENTS SPECS FLOWS

Connect Git Console Terminal

Globals Vault Tools

23:11 18-03-2026

uppu snehanjali's Workspace

Search

Invites Upgrade

PUT http: DEL http: GET http: POST http: GET http: PUT http: DEL http: GET http: POST http: GET http: + No enviro...

http://localhost:3000/students

GET http://localhost:3000/students

Send

Docs Params Authorization Headers 6 Body Scripts Tests Settings Cookies

Query Params

Key	Value	Description	Bulk Edit
Key	Value	Description	

Body Cookies Headers 7 Test Results

200 OK · 4 ms · 301 B

JSON Preview Visualize

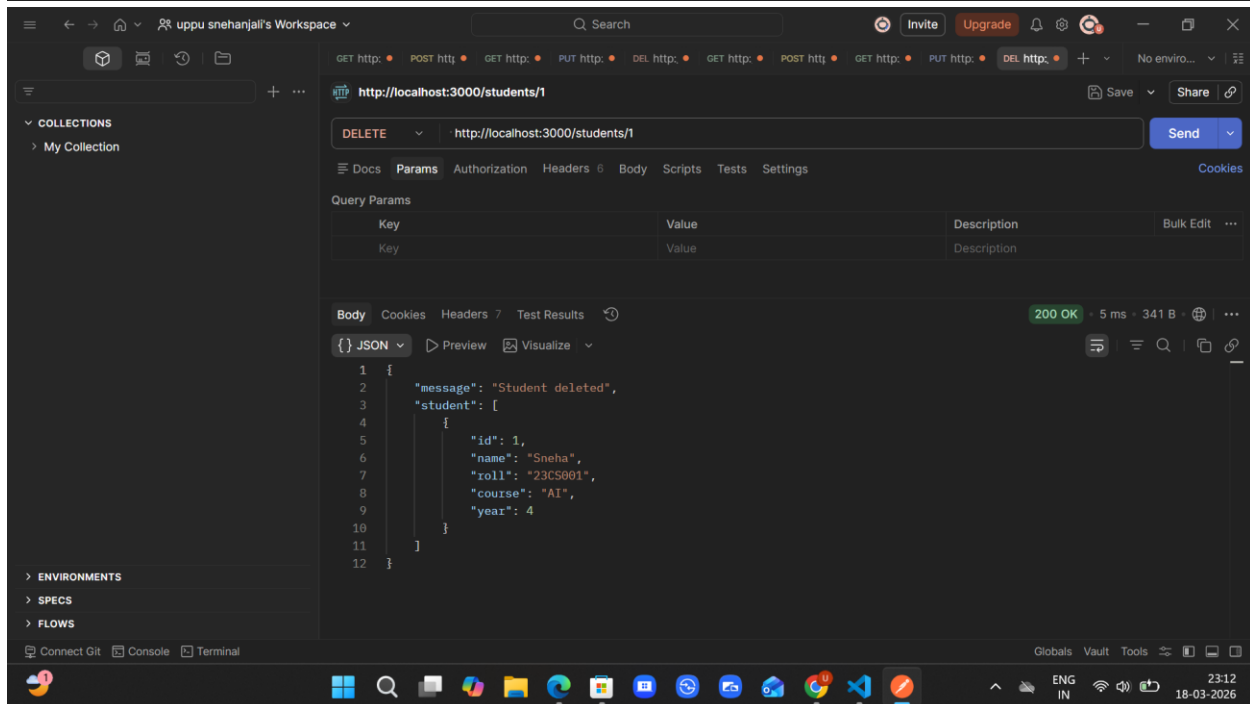
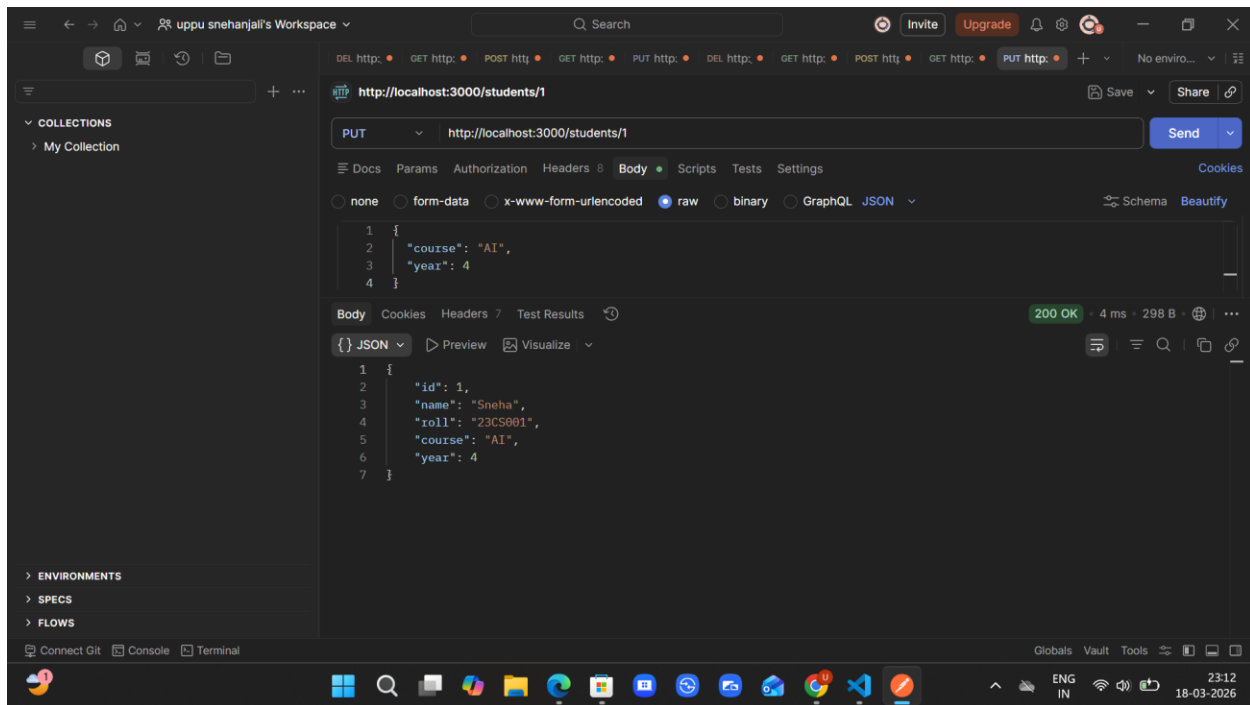
```
1 [
2   {
3     "id": 1,
4     "name": "Sneha",
5     "roll": "23CS001",
6     "course": "CSE",
7     "year": 3
8   }
9 ]
```

ENVIRONMENTS SPECS FLOWS

Connect Git Console Terminal

Globals Vault Tools

23:11 18-03-2026



Explanation:

-->This task is used to manage student details using an API.

-->New students can be added using the POST method.

-->Student data can be viewed using the GET method.

-->Details can be updated using PUT and deleted using DELETE.

-->It helps to understand CRUD operations in real-world applications.